Project Plan

Study Buddy – Smart Study Material Generator

Team 25

Role	Name
Industry Partner	N/A
Primary Instructor	Prof. [Name]
Team Lead / Full Stack Developer	Parsa Molahosseini
Backend & Integration Developer	Kevin George Buhain
QA & Documentation Lead	Mehrad Bayat
UI/UX Designer	Jerry-Lee Somera

Document Revision History

Revision #	Date	Changes
1.0	October 6, 2025	Initial project plan release

---

1. Executive Summary

Study Buddy is a web-based platform that lets students upload their notes (PDF/TXT) and automatically generate flashcards, quizzes, and summaries. It improves study efficiency through automation, progress tracking, and spaced-review reminders. The platform aims to generate study materials 80% faster than manual creation, helping students optimize their learning workflow.

Item	Details
Objective	Develop an automated study material generator to help students create flashcards, quizzes, and summaries from uploaded notes
Corporate Goals Addressed	Enhance student learning efficiency; Provide accessible educational technology tools; Reduce manual study material preparation time
Planned Start Date	September 2025
Planned End Date	March 2026

---

2. Project Approvers, Reviews and Distribution List

Project Role	Name	E-mail	Date
Supervisor	Prof. [Name]	[email@georgebrown.ca]	
Team Lead / Full Stack Developer	Parsa Molahosseini	parsa.molahosseini@georgebrown.ca	
Backend & Integration Developer	Kevin George Buhain	kevin.buhain@georgebrown.ca	
QA & Documentation Lead	Mehrad Bayat	mehrad.bayat@georgebrown.ca	
UI/UX Designer	Jerry-Lee Somera	jerry-lee.somera@georgebrown.ca	

---

3. Scope

In Scope	Out of Scope
File upload & text extraction (PDF/TXT)	AI integration (reserved for Phase 2)
Flashcard generation with customization	Native mobile applications (iOS/Android)
Quiz generation with multiple choice format	LMS integration (Canvas, Moodle, Blackboard)
Progress dashboard and analytics	Video content processing
Smart highlight extraction from notes	Handwriting recognition
Spaced-review reminders via email/notifications	Real-time collaboration features

---

4. Deliverables

Deliverable	Description
Functional Web Application	Working prototype with all core features implemented (file upload, flashcard generation, quiz creation, dashboard)
Source Code Repository	Complete codebase with version control history on GitHub, including README and setup instructions
Documentation Package	Project summary, vision document, high-level requirements (HLR), user manual, and technical documentation
Final Presentation	Demo slide deck showcasing features, architecture, and project outcomes

---

5. Assumptions

This project makes the following assumptions:

- Students have access to digital notes in PDF or TXT format

- Users have reliable Internet access and use modern web browsers
- Target users are comfortable with web-based applications and basic file uploads
- Study materials provided by users are primarily in English language
- Users are willing to create accounts for progress tracking features
- The hosting platform (Heroku/Render) will provide sufficient free-tier resources for prototype development

---

6. Dependencies

The following are the internal and external dependencies that will have to be acknowledged and addressed:

Technical Dependencies:

- PyPDF2 or Apache Tika for PDF text extraction functionality
- Chart.js library for data visualization in progress dashboard
- FastAPI framework for backend REST API development
- PostgreSQL database for persistent data storage
- Heroku or Render cloud platform for application deployment
- GitHub for version control and team collaboration

Team Dependencies:

- Backend API completion required before frontend integration can begin
- UI mockups and design approval needed before implementation phase
- Regular team availability for weekly sprint meetings and code reviews
- Access to university network/resources for testing and development

External Dependencies:

- Availability of free cloud hosting services
- Stable internet connectivity for collaborative development
- Open-source library maintenance and compatibility

---

7. Risk Management

Potential Risk	Severity (H/M/L)	Likelihood (H/M/L)	Management Strategy
Parsing errors with complex PDF formats	M	L	Validate file types on upload; implement robust error handling; test with diverse file formats early; provide clear error messages to users
Time overrun on core features	H	M	Follow strict sprint planning with 2-week iterations; prioritize MVP features first; maintain 1-week buffer time in schedule; weekly progress reviews
Data loss or corruption	M	L	Implement regular automated backups (daily); use version control for all code; maintain database redundancy; implement proper transaction handling
Team member availability issues	M	M	Cross-train team members on multiple components; maintain comprehensive documentation; redistribute tasks as needed; hold regular knowledge-sharing sessions
Browser compatibility issues	M	L	Test on major browsers (Chrome, Firefox, Safari, Edge); use standard web APIs and CSS; implement graceful degradation for older browsers
Database performance with large files	M	M	Implement file size limits (max 10MB initially); optimize database queries with indexing; use pagination for large datasets; implement caching strategies

---

8. Communication

Reporting

The following reports will be produced:

Report	Audience	Frequency
Sprint Progress Report	Team & Instructor	Weekly (Mondays)
Milestone Status Update	Instructor	Bi-weekly (end of sprint)
Technical Documentation	Team & Stakeholders	As completed per milestone
Risk & Issue Log	Team & Instructor	Weekly (updated during team meetings)

Meetings

The following meetings/communication will be established:

Meeting	Purpose	Attendees	Frequency
Team Sprint Meeting	Review progress, plan upcoming tasks, resolve blockers, assign new work	All team members	Weekly (Mondays, 6:00 PM)
Professor Check-in	Project status update, milestone review, receive guidance and feedback	Team lead + Instructor	Bi-weekly (Fridays)
Code Review Sessions	Quality assurance, knowledge sharing, ensure coding standards	Development team	As needed (minimum bi-weekly)
Sprint Retrospective	Reflect on process, discuss improvements, celebrate wins	All team members	End of each 2-week sprint

---

9. Task Listing (WBS - Work Breakdown Structure)

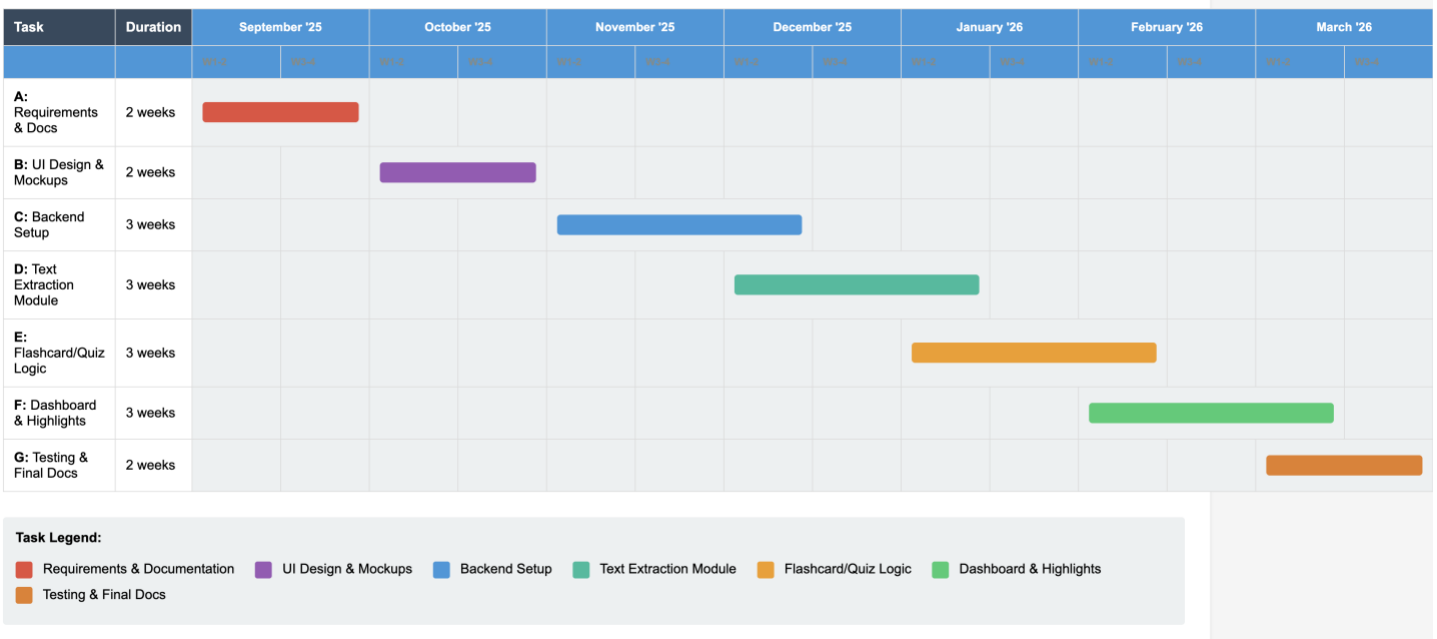
Reference	Tasks	Duration	Dependency
A	Requirements gathering & documentation (vision doc, HLR, user stories)	2 weeks	–
B	UI design & mockups creation (wireframes, user flow diagrams)	2 weeks	A
C	Backend setup (FastAPI framework, PostgreSQL database, API endpoints)	3 weeks	B
D	Text extraction module development (PDF/TXT parsing, error handling)	3 weeks	C
E	Flashcard & quiz generation logic (content parsing, question creation)	3 weeks	D
F	Dashboard & highlights implementation (analytics, progress tracking)	3 weeks	E
G	Testing, debugging & final documentation (user manual, bug fixes)	2 weeks	F

Total Project Duration: 18 weeks (approximately 4.5 months)

---

10. Gantt Chart

INSERT YOUR GANTT CHART IMAGE HERE



The Gantt chart above visualizes the project timeline from September 2025 through March 2026, showing task dependencies and parallel work streams.

---

11. Milestones

Major Activity or Milestone	Estimated Milestone Target Date	Owner/Reviewer Team Members
Project Proposal Approved	September 25, 2025	Team 25
Major Activity or Milestone	**Estimated Milestone Target Date**	**Owner/Reviewer Team Members**
UI Design & Backend Setup Ready	November 15, 2025	Parsa, Kevin
Text Extraction Module Complete	December 15, 2025	Parsa, Kevin
MVP (Minimum Viable Product) Completed	January 25, 2026	Team 25
Final Testing & QA Completed	February 20, 2026	Mehrad
Final Demo & Presentation	March 10, 2026	Jerry-Lee

---

12. RAM – Responsibility Assignment Matrix

Task	Parsa	Kevin	Mehrad	Jerry-Lee
Architecture & Backend Development	A	R	C	I
Integration & System Testing	R	A	C	I
UI Design & Frontend Development	I	C	R	A
QA & Documentation	C	I	A	R
Project Planning & Coordination	A	C	I	R
Database Design & Implementation	A	R	I	C
Text Extraction Module	R	A	C	I
Deployment & DevOps	C	R	A	I

Legend:

- **A** = Accountable (ultimate ownership and final approval)
- **R** = Responsible (does the work to complete the task)
- **C** = Consulted (provides input and expertise)
- **I** = Informed (kept updated on progress)

---

13. Approval

The signatures below indicate their approval of the contents of this document.

Project Role	Name	Signature	Date
Supervisor	Prof. [Name]	_____	
Team Lead / Full Stack Developer	Parsa Molahosseini	_____	
Backend & Integration Developer	Kevin George Buhain	_____	
QA & Documentation Lead	Mehrad Bayat	_____	
UI/UX Designer	Jerry-Lee Somera	_____	

End of Document

Appendix A: Success Criteria

The project will be considered successful when:

- Users can successfully upload PDF/TXT files and receive generated flashcards
- Quiz generation functionality produces valid multiple-choice questions
- Dashboard displays accurate progress metrics and analytics
- System maintains 95%+ uptime during testing phase
- All core features pass user acceptance testing
- Documentation is complete and accessible

Appendix B: Technology Stack Summary

Component	Technology
Backend Framework	FastAPI (Python)
Database	PostgreSQL
Frontend	HTML5, CSS3, JavaScript
Data Visualization	Chart.js
Text Extraction	PyPDF2 / Apache Tika
Hosting	Heroku or Render
Version Control	GitHub

Document prepared by Team 25

Last updated: October 6, 2025